TARGET BUSINESS CASE STUDY PROJECT-SQL/BIGQUERY

Shantanu Rawat | Shantanu1tech@gmail.com | +91 9310315068

www.linkedin.com/in/shantanu-rawat

https://github.com/Shantanu-Rawat-Techno

Context:

Target is a globally renowned brand and a prominent retailer in the United States. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.

This particular business case focuses on the operations of Target in Brazil and provides insightful information about 100,000 orders placed between 2016 and 2018. The dataset offers a comprehensive view of various dimensions including the order status, price, payment and freight performance, customer location, product attributes, and customer reviews.

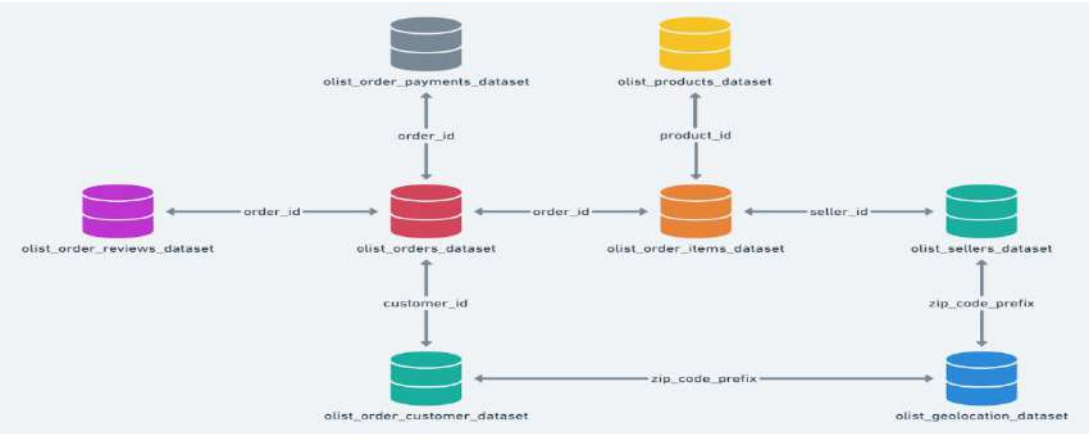
By analysing this extensive dataset, it becomes possible to gain valuable insights into Target's operations in Brazil. The information can shed light on various aspects of the business, such as order processing, pricing strategies, payment and shipping efficiency, customer demographics, product characteristics, and customer satisfaction levels.

Dataset: https://drive.google.com/drive/folders/1TGEc66YKbD443nslRi1bWgVd238gJCnb

The data is available in 8 csv files:

1. customers.csv
2. sellers.csv
3. order_items.csv
4. geolocation.csv
5. payments.csv
6. reviews.csv
7. orders.csv
8. products.csv

Dataset schema:



Problem Statement:

Assuming you are a data analyst/ scientist at Target, you have been assigned the task of analysing the given dataset to extract valuable insights and provide actionable recommendations.

What does 'good' look like?

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1.1 Data type of all columns in the "customers" table.

```
SELECT column_name, data_type
FROM target-sql-488119.TARGET_SQL.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'customers';
```

Query results

+ Create conversation | Save results | Open in

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

INSIGHTS:

Most fields are **STRING type**, indicating the dataset is primarily categorical and ID-based rather than numerical or transactional. Only customer_zip_code_prefix is numeric (INT64), suggesting limited quantitative analysis unless combined with other datasets.

1.2 Get the time range between which the orders were placed.

```
SELECT
MIN(order_purchase_timestamp) AS first_order_date,
MAX(order_purchase_timestamp) AS last_order_date
FROM target-sql-488119.TARGET_SQL.orders;
```

Job information | Results | Visualisation | JSON

Row	first_order_date	last_order_date
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

INSIGHTS:

The order history spans from September 4, 2016, to October 17, 2018, with a total time span of 2 years, 1 month, and 12 days.

1.3 Count the Cities & States of customers who ordered during the given period.

```sql
SELECT
COUNT(DISTINCT c.customer_city) AS count_of_cities,
COUNT(DISTINCT c.customer_state) AS count_of_states
FROM target-sql-488119.TARGET_SQL.orders o
JOIN target-sql-488119.TARGET_SQL.customers c
USING (customer_id);
```

Job information | Results | Visualisation

Row	count_of_cities	count_of_states
1	4119	27

INSIGHTS:

- The dataset covers **4,119 cities across 27 states**, indicating wide geographic distribution.
- With 27 states, the customer base appears **diverse and nationally spread**.

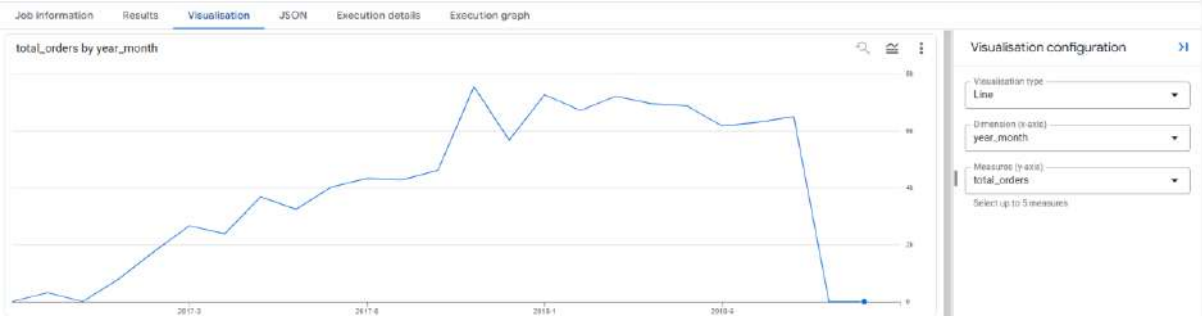
2. In-depth Exploration:

2.1 Is there a growing trend in the no. of orders placed over the past years?

```
SELECT
EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
CONCAT( EXTRACT(YEAR FROM order_purchase_timestamp), '-', EXTRACT(MONTH FROM
order_purchase_timestamp) ) AS year_month,
COUNT(order_id) AS total_orders
FROM target-sql-488119.TARGET_SQL.orders
GROUP BY year, month, year_month
ORDER BY year, month, year_month;
```

Job information | **Results** | Visualisation | JSON | Execution details | Execution

Row	year	month	year_month	total_orders
1	2016	9	2016-9	4
2	2016	10	2016-10	324
3	2016	12	2016-12	1
4	2017	1	2017-1	800
5	2017	2	2017-2	1780
6	2017	3	2017-3	2682
7	2017	4	2017-4	2404
8	2017	5	2017-5	3700
9	2017	6	2017-6	3245
10	2017	7	2017-7	4026



INSIGHTS:

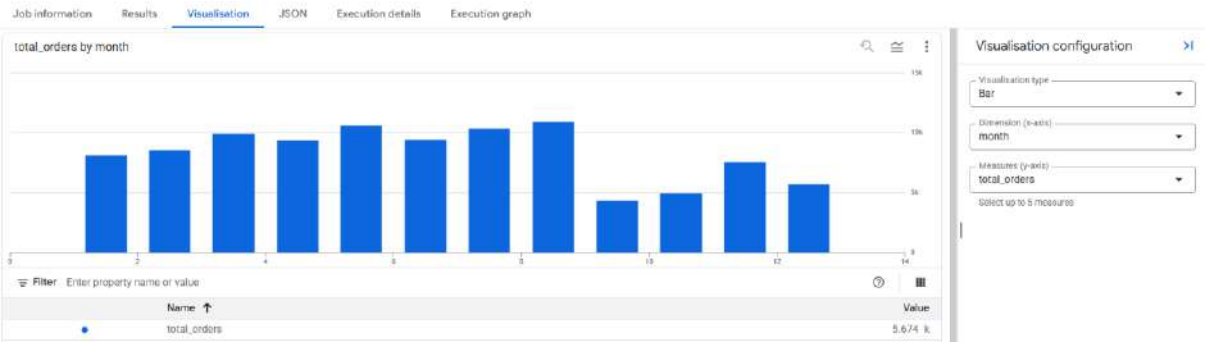
- Orders show **strong growth throughout 2017**, peaking in **November 2017 (7,544 orders)**, indicating possible seasonal demand (e.g., holiday sales).
- 2018 maintains consistently high volumes (~6,000–7,200/month) until a **sharp drop in Sept–Oct 2018**, likely due to incomplete data for those months.
- The number of orders generally shows a strong upward trend from late 2016 through 2018, despite some natural monthly fluctuations likely driven by seasonality or external factors.

2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

```
SELECT
EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
COUNT(order_id) AS total_orders
FROM target-sql-488119.TARGET_SQL.orders
GROUP BY month
ORDER BY month;
```

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	month	total_orders
1	1	8069
2	2	8508
3	3	9893
4	4	9343
5	5	10573
6	6	9412
7	7	10318
8	8	10843
9	9	4305
10	10	4959
11	11	7544
12	12	5674



INSIGHTS:

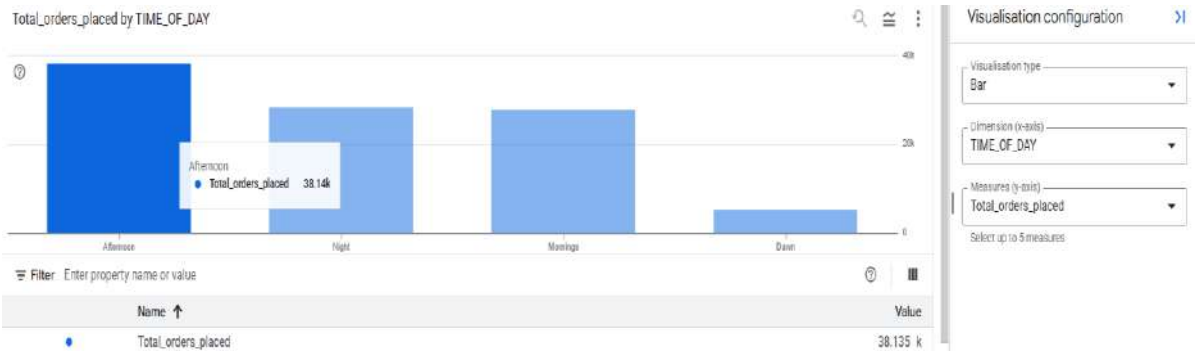
- Orders peak during the **summer months (May to August)**, consistently exceeding 9,000 per month, indicating strong seasonal demand.
- There is a noticeable **decline in September and October**, followed by a moderate recovery in November and a drop in December.

2.3 During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

0-6 hrs : Dawn, 7-12 hrs : Mornings, 13-18 hrs : Afternoon. 19-23 hrs : Night

```
SELECT
CASE
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN 'Dawn'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN 'Mornings'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN 'Afternoon'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23 THEN 'Night'
END AS TIME_OF_DAY,
count (order_id) as Total_orders_placed
FROM target-sql-488119.TARGET_SQL.orders
group by TIME_OF_DAY
order by Total_orders_placed desc
```

Row	TIME_OF_DAY	Total_orders_plac...
1	Afternoon	38135
2	Night	28331
3	Mornings	27733
4	Dawn	5242



INSIGHTS:

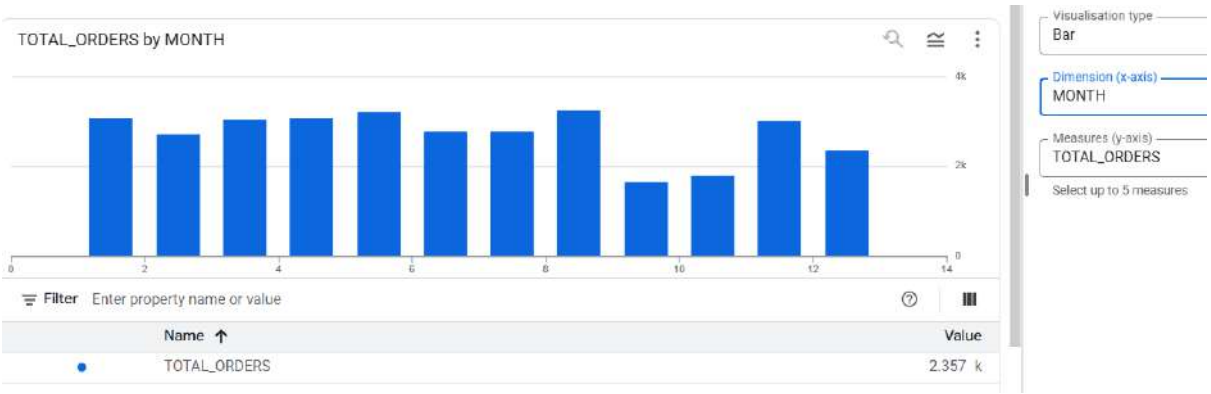
Brazilian customers mostly place their orders in the **Afternoon (13-18 hrs)**, followed by Night and Mornings, with the fewest orders occurring at Dawn.

3. Evolution of E-commerce orders in the Brazil region:

3.1 Get the month on month no. of orders placed in each state.

```
SELECT
C.customer_state AS STATE,
EXTRACT(YEAR FROM O.order_purchase_timestamp) AS YEAR,
EXTRACT(MONTH FROM O.order_purchase_timestamp) AS MONTH,
COUNT(O.order_id) AS TOTAL_ORDERS
FROM target-sql-488119.TARGET_SQL.orders O JOIN target-sql-488119.TARGET_SQL.customers C
USING(customer_id)
GROUP BY STATE, YEAR, MONTH
ORDER BY STATE, YEAR, MONTH
```

Row	STATE	YEAR	MONTH	TOTAL_ORDERS
1	AC	2017	1	2
2	AC	2017	2	3
3	AC	2017	3	2
4	AC	2017	4	5
5	AC	2017	5	8
6	AC	2017	6	4
7	AC	2017	7	5
8	AC	2017	8	4
9	AC	2017	9	5
10	AC	2017	10	6



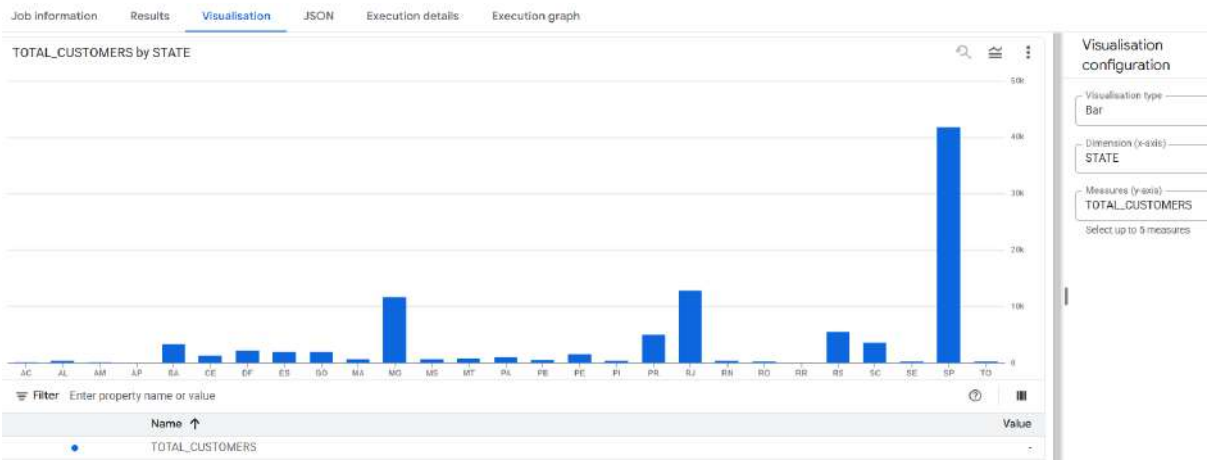
INSIGHTS:

- Orders peak in **October**, followed by **May**, indicating strong seasonal demand during these months.
- **September** records the lowest order volume, highlighting a potential demand dip or off-season period.

3.2 How are the customers distributed across all the states?

```
SELECT
customer_state AS STATE, COUNT (customer_id) AS TOTAL_CUSTOMERS
FROM target-sql-488119.TARGET_SQL.customers
GROUP BY customer_state
ORDER BY customer_state
```

Row	STATE	TOTAL_CUSTOM...
1	AC	81
2	AL	413
3	AM	148
4	AP	68
5	BA	3380
6	CE	1336
7	DF	2140
8	ES	2033
9	GO	2020
10	MA	747



INSIGHTS:

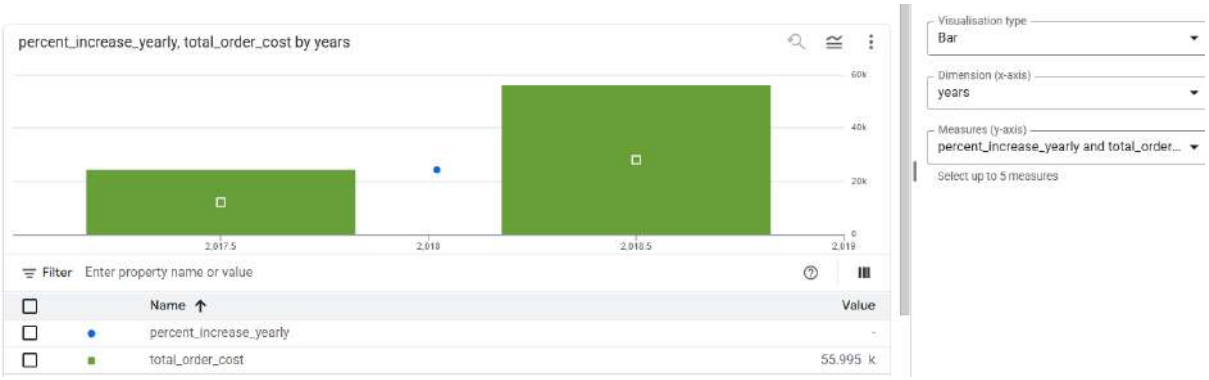
- Customer distribution is highly concentrated in **SP (41,746)**, followed by **RJ (12,852)** and **MG (11,635)**, indicating these states dominate the customer base.
- In contrast, states like **RR (46)**, **AC (81)**, and **AP (68)** have minimal customer presence, reflecting significant regional imbalance in customer distribution.

4. **Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.**

4.1 Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

```
WITH cte AS (
    SELECT
        EXTRACT(YEAR FROM o.order_purchase_timestamp) AS years,
        COUNT(p.payment_value) AS total_order_cost
    FROM `target-sql-488119.TARGET_SQL.orders` o
    JOIN `target-sql-488119.TARGET_SQL.payments` p
        USING (order_id)
    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) IN (2017, 2018)
      AND EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8
    GROUP BY years
)

SELECT
    years,
    total_order_cost,
    ROUND(
        (
            (total_order_cost - LAG(total_order_cost) OVER (ORDER BY years))
            / LAG(total_order_cost) OVER (ORDER BY years)
        ) * 100,
    2) AS percent_increase_yearly
FROM cte
ORDER BY years;
```

Row	years	total_order_cost	percent_increase...
1	2017	24391	null
2	2018	55995	129.57



INSIGHTS:

- The total order cost increased from **24,391 in 2017 (Jan–Aug)** to **55,995 in 2018 (Jan–Aug)**, reflecting a significant **129.57% year-on-year growth**.
- This sharp rise indicates strong revenue expansion in 2018 compared to the same period in 2017.

4.2 Calculate the Total & Average value of order price for each state.

```
select
  c.customer_state as state,
  round(sum(oi.price),2) as total_value_of_order_price,
  round(sum(oi.price)/count(distinct oi.order_id),2) as avg_value_of_order_price

from target-sql-488119.TARGET_SQL.customers c join target-sql-488119.TARGET_SQL.orders o
using (customer_id) join
target-sql-488119.TARGET_SQL.order_items oi using (order_id)

group by state
```

Row	state	total_value_of_or...	avg_value_of_ord...
1	AC	15982.95	197.32
2	AL	80314.81	195.41
3	AM	22356.84	152.09
4	AP	13474.3	198.15
5	BA	511349.99	152.28
6	CE	227254.71	171.25
7	DF	302603.94	142.4
8	ES	275037.31	135.82
9	GO	294591.95	146.78
10	MA	119648.22	161.69

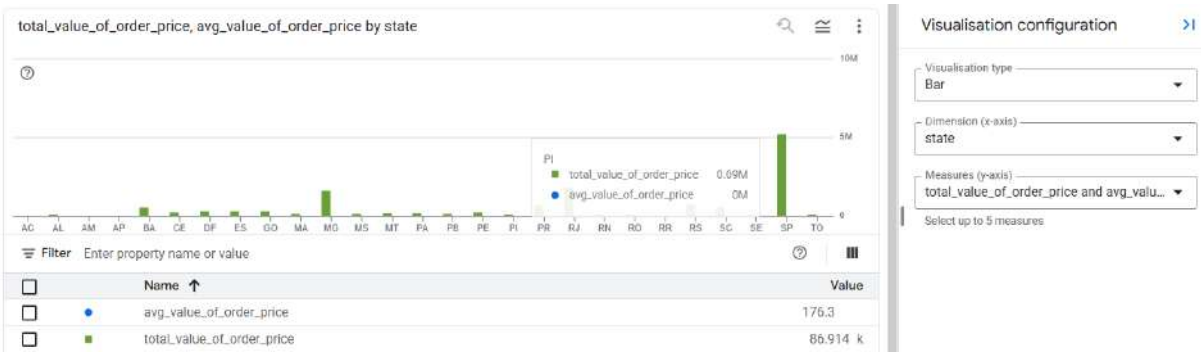
INSIGHTS:

- **SP (5.20M)**, **RJ (1.82M)**, and **MG (1.58M)** generate the highest total order values, indicating they are the primary revenue-driving states.
- However, **PB (216.67)** and **AP (198.15)** have the highest average order values, suggesting stronger per-order spending despite lower overall volumes compared to SP and RJ.

4.3 Calculate the Total & Average value of order freight for each state.

```
select
  c.customer_state as state,
  round(sum(oi.freight_value),2) as total_freightvalue,
  round(sum(oi.freight_value)/count(distinct oi.order_id),2) as avg_freightvalue

from target-sql-488119.TARGET_SQL.customers c join target-sql-488119.TARGET_SQL.orders o
using (customer_id) join
target-sql-488119.TARGET_SQL.order_items oi using (order_id)

group by state
```

Row	state	total_freightvalue	avg_freightvalue
1	AC	3686.75	45.52
2	AL	15914.59	38.72
3	AM	5478.89	37.27
4	AP	2788.5	41.01
5	BA	100156.68	29.83
6	CE	48351.59	36.44
7	DF	50625.5	23.82
8	ES	49764.6	24.58
9	GO	53114.98	26.46
10	MA	31523.77	42.6



INSIGHTS:

- **SP (718,723)**, **RJ (305,589)**, and **MG (270,853)** incur the highest total freight costs, driven by their large order volumes.
- However, **RR (48.59)**, **PB (48.35)**, and **RO (46.22)** show the highest average freight per order, indicating relatively higher shipping cost per transaction compared to high-volume states like **SP (17.37)**.

5.Analysis based on sales, freight and delivery time.

5.1 Find the no. of days taken to deliver each order from the order's purchase date as delivery time. Also, calculate the difference (in days) between the estimated & actual delivery date of an order. Do this in a single query.

```
SELECT
order_id,
DATE_DIFF(DATE(order_delivered_customer_date),  DATE(order_purchase_timestamp),  DAY)  AS
days_to_deliver,
DATE_DIFF(DATE(order_delivered_customer_date), DATE(order_estimated_delivery_date), DAY) AS
diff_estimated_delivery
FROM `target-sql-488119.TARGET_SQL.orders`
ORDER BY days_to_deliver desc, diff_estimated_delivery desc;
```

Row	order_id	days_to_deliver	diff_estimated_d...
1	ca07593549f1816d26a572e06...	210	181
2	1b3190b2dfa9d789e1f14c05b6...	208	188
3	440d0d17af552815d15a9e41a...	196	165
4	285ab9426d6982034523a855f...	195	166
5	2fb597c2f772eca01b1f5c561bf...	195	155
6	0f4519c5f1c541ddec9f21b3bd...	194	161
7	47b40429ed8cce3aee9199792...	191	175
8	2fe324febf907e3ea3f2aa96508...	190	167
9	c27815f7e3dd0b926b5855262...	188	162
10	2d7561026d542c8dbd8f0daea...	188	159

INSIGHTS:

Several orders show extreme delays(over 200 days), significantly exceeding estimated delivery dates, indicating major gaps in delivery planning and execution.

5.2 Find out the top 5 states with the highest & lowest average freight value.

```sql
WITH state_avg AS (
    SELECT
        c.customer_state,
        ROUND(AVG(oi.freight_value), 2) AS avg_freight
    FROM `target-sql-488119.TARGET_SQL.customers` c
    JOIN `target-sql-488119.TARGET_SQL.orders` o
        ON c.customer_id = o.customer_id
    JOIN `target-sql-488119.TARGET_SQL.order_items` oi
        ON o.order_id = oi.order_id
    GROUP BY c.customer_state
)
SELECT * FROM (
    SELECT 'Highest' AS category, customer_state, avg_freight
    FROM state_avg
    ORDER BY avg_freight DESC
    LIMIT 5
)
UNION ALL
SELECT * FROM (
    SELECT 'Lowest' AS category, customer_state, avg_freight
    FROM state_avg
    ORDER BY avg_freight ASC
    LIMIT 5
);
```

Row	category	customer_state	avg_freight
1	Highest	RR	42.98
2	Highest	PB	42.72
3	Highest	RO	41.07
4	Highest	AC	40.07
5	Highest	PI	39.15
6	Lowest	DF	21.04
7	Lowest	RJ	20.96
8	Lowest	MG	20.63
9	Lowest	PR	20.53
10	Lowest	SP	15.15

INSIGHTS:

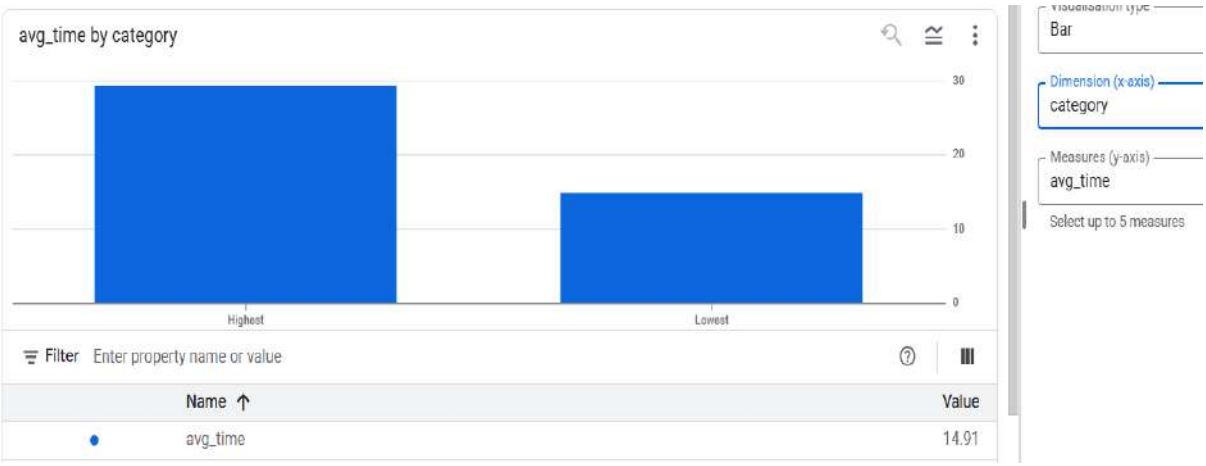
- **RR, PB, and RO** have the highest average freight costs, indicating relatively expensive shipping per order in these states.
- In contrast, **SP, PR, and MG** record the lowest average freight values, suggesting better logistics efficiency or cost advantages in high-volume regions.

5.3 Find out the top 5 states with the highest & lowest average delivery time.

```
WITH state_avg AS (
    SELECT
        c.customer_state,
                                ROUND(avg(DATE_DIFF(DATE(o.order_delivered_customer_date),
DATE(o.order_purchase_timestamp), DAY)), 2) AS avg_time
            FROM `target-sql-488119.TARGET_SQL.customers` c
    JOIN `target-sql-488119.TARGET_SQL.orders` o
        ON c.customer_id = o.customer_id

    GROUP BY c.customer_state
)

SELECT * FROM (
    SELECT 'Highest' AS category, customer_state, avg_time
    FROM state_avg
    ORDER BY avg_time DESC
    LIMIT 5
)

UNION ALL

SELECT * FROM (
    SELECT 'Lowest' AS category, customer_state, avg_time
    FROM state_avg
    ORDER BY avg_time ASC
    LIMIT 5
);
```

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	category	customer_state	avg_time
1	Highest	RR	29.34
2	Highest	AP	27.18
3	Highest	AM	26.36
4	Highest	AL	24.5
5	Highest	PA	23.73
6	Lowest	SP	8.7
7	Lowest	PR	11.94
8	Lowest	MG	11.95
9	Lowest	DF	12.9
10	Lowest	SC	14.91



INSIGHTS:

- **RR, AP, and AM** have the highest average delivery times, indicating slower fulfillment performance in these states.
- Conversely, **SP, PR, and MG** show the lowest average delivery times, reflecting faster and more efficient delivery operations.

5.4 Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
(can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.)

```
SELECT
  c.customer_state,
  ROUND(AVG(
    DATE_DIFF(
      DATE(o.order_delivered_customer_date),
      DATE(o.order_estimated_delivery_date),
      DAY
    )
  ), 2) AS avg_delivery_diff
FROM `target-sql-488119.TARGET_SQL.customers` c
JOIN `target-sql-488119.TARGET_SQL.orders` o
  ON c.customer_id = o.customer_id
WHERE o.order_delivered_customer_date IS NOT NULL
 AND o.order_estimated_delivery_date IS NOT NULL
GROUP BY c.customer_state
ORDER BY avg_delivery_diff ASC
LIMIT 5;
```

Row	customer_state	avg_delivery_diff
1	AC	-20.73
2	RO	-20.1
3	AP	-19.69
4	AM	-19.57
5	RR	-17.29

INSIGHTS:

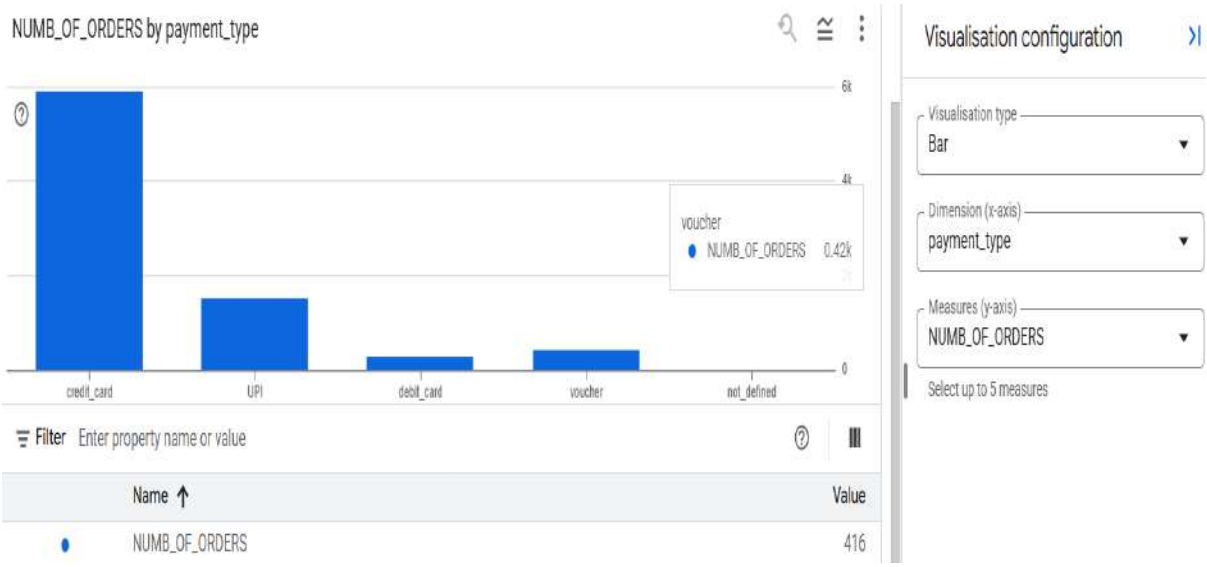
- **AC, RO, and AP** show the most negative average delivery differences, meaning orders are delivered significantly earlier than the estimated date.
- This indicates strong delivery efficiency and better-than-promised fulfillment performance in these states.

6. Analysis based on the payments:

6.1 Find the month on month no. of orders placed using different payment types.

```
SELECT
  extract(year from order_purchase_timestamp) as year,
  extract(month from order_purchase_timestamp) as month, payment_type,
  count(order_id) AS NUMB_OF_ORDERS

FROM  target-sql-488119.TARGET_SQL.orders  o  join  target-sql-488119.TARGET_SQL.payments  p
using(order_id)

group by year,month, payment_type
order by year,month, payment_type
```

Row	year	month	payment_type	NUMB_OF_ORDE...
1	2016	9	credit_card	3
2	2016	10	UPI	63
3	2016	10	credit_card	254
4	2016	10	debit_card	2
5	2016	10	voucher	23
6	2016	12	credit_card	1
7	2017	1	UPI	197
8	2017	1	credit_card	583
9	2017	1	debit_card	9
10	2017	1	voucher	61



INSIGHTS:

- **Credit Card** is the dominant payment mode across all months, showing consistent and strong growth, especially peaking in late 2017 and early 2018.
- **UPI** shows steady adoption growth over time, while **debit card and voucher** transactions remain comparatively low, indicating clear customer preference toward credit-based payments.

6.2 Find the no. of orders placed on the basis of the payment installments that have been paid.

```
SELECT
    p.payment_installments,
    COUNT(DISTINCT o.order_id) AS num_of_orders
FROM `target-sql-488119.TARGET_SQL.orders` o
JOIN `target-sql-488119.TARGET_SQL.payments` p
    ON o.order_id = p.order_id
WHERE p.payment_installments > 0
GROUP BY p.payment_installments
ORDER BY p.payment_installments;
```

Row	payment_installm...	num_of_orders
1	1	49060
2	2	12389
3	3	10443
4	4	7088
5	5	5234
6	6	3916
7	7	1623
8	8	4253
9	9	644
10	10	5315

INSIGHTS:

- The majority of orders (**49,060**) are paid in **1 installment**, showing strong customer preference for full upfront payments.
- Orders sharply decline as installments increase, with very few transactions beyond 12 installments, indicating limited adoption of long-term payment plans.

THANK YOU

Shantanu Rawat | Shantanu1tech@gmail.com | +91 9310315068

www.linkedin.com/in/shantanu-rawat

https://github.com/Shantanu-Rawat-Techno